Les concepts de programmation procédurale

Arfaoui Ahlem

Les fonctions(1):

Principe:

- Jusqu'ici, nous avons résolu nos problèmes à l'aide de fonctions prédéfinies et d'une seule fonction: **la fonction principale main().**
- Pour des problèmes plus complexes, nous obtenons ainsi de longues listes d'instructions, peu structurées et par conséquent peu compréhensibles. En plus, il faut souvent répéter les mêmes suites de commandes dans le programme.



- La plupart des langages de programmation nous permettent de subdiviser nos programmes en sous-programmes, fonctions ou procédures plus simples et plus compacts. A l'aide de ces structures nous pouvons **modulariser** nos programmes pour obtenir des solutions plus élégantes et plus efficientes.
- Une **fonction** désigne une entité de données et d'instructions qui fournissent une solution à une (petite) partie bien définie d'un **problème** plus **complexe**.
- Une **fonction** peut faire appel à d'autres fonctions, leur transmettre des données et recevoir des données en retour.

Les fonctions(2):

Avantages:

- Voici quelques avantages d'un programme modulaire:

* **Meilleure lisibilité**
* **Diminution du risque d'erreurs**
* **Possibilité de tests sélectifs**
* **Réutilisation de modules déjà existants**

Il est facile d'utiliser des modules qu'on a écrits soi-même ou qui ont été développés par d'autres personnes.

* **Simplicité de l'entretien**

Un module peut être changé ou remplacé sans devoir toucher aux autres modules du programme.

* **Favorisation du travail en équipe**

Un programme peut être développé en équipe par délégation de la programmation des modules à différentes personnes ou groupes de personnes. Une fois développés, les modules peuvent constituer une base de travail commune.

Les fonctions(3):

Avantages:

* **Hiérarchisation des modules**

Un programme peut d'abord être résolu globalement au niveau du module principal. Les détails peuvent être reportés à des modules sous-ordonnés qui peuvent eux aussi être subdivisés en sous-modules et ainsi de suite. De cette façon, nous obtenons une hiérarchie de modules.

Les fonctions(4):

Définition d'une fonction:

Dans la définition d'une fonction, nous indiquons:

-Le **nom** de la fonction

- Le **type**, le **nombre** et les **noms** des paramètres de la fonction
- le **type du résultat** fourni par la fonction
- les **données locales** à la fonction
- les instructions à exécuter

Syntaxe:

```
Type_resultat NomFonction (Typearg1 Nomarg1, Typearg2 Nomarg2, ... )
{
déclarations locales
instructions
}
```

Les fonctions(5):

Définition d'une fonction:

Exemple:

La fonction **MAX** est du type **int** et elle a besoin de **deux paramètres** du type **int**. Le résultat de la fonction **MAX** peut être intégré dans d'autres expressions.

```
int MAX (int N1, int N2)
 {
if  (N1>N2)
 return N1;
else
 return N2;
 }
```

Les fonctions(6):

Définition d'une fonction:

Remarques:

•Une fonction peut fournir comme résultat:

- un type **standard** (int ,float,...),
- une **structure** (définie par **struct** - pas traité dans ce cours),
- une **réunion** (définie par **union** - pas traité dans ce cours),
- un **pointeur**,
- **void** (la fonction correspond alors à une 'procédure').

•Une fonction **ne peut pas fournir** comme résultat **des tableaux**, **des chaînes de caractères ou des fonctions**. (**Attention:** Il est cependant possible de renvoyer un pointeur sur le premier élément d'un tableau ou d'une chaîne de caractères.)

Les fonctions(7):

Déclaration d'une fonction:

- En C, il faut déclarer chaque fonction avant de pouvoir l'utiliser.
- La déclaration informe le compilateur du type des paramètres et du résultat de la fonction. ➔ A l'aide de ces données, le compilateur peut contrôler si le nombre et le type des paramètres d'une fonction sont corrects.

Prototype d'une fonction:

La déclaration d'une fonction se fait par **un prototype** de la fonction qui indique uniquement le **type des données** transmises et reçues par la fonction.

Déclaration : Prototype d'une fonction

Type_resultat Nom_Fonction (**TypeArg1**, **TypeArg2**, ...);
ou bien
Type_resultat Nom_Fonction (**TypeArg1** NomArg1, **TypeArg2** NomArg2, ...);

Les fonctions(8):

Déclaration d'une fonction:

Attention: Lors de la déclaration, **le nombre** et **le type** des paramètres doivent nécessairement correspondre à ceux de la définition de la fonction.

- On peut facultativement inclure les noms des paramètres dans la déclaration, mais ils ne sont pas considérés par le compilateur. Les noms fournissent pourtant une information intéressante pour le programmeur qui peut en déduire le rôle des différents paramètres.

Règles pour la déclaration des fonctions

De façon analogue aux déclarations de variables, nous pouvons déclarer une fonction localement ou globalement. La définition des fonctions joue un rôle spécial pour la déclaration. En résumé, nous allons considérer les règles suivantes:

Déclaration locale:

Une fonction peut être déclarée localement **dans la fonction qui l'appelle** (**avant la déclaration des variables**). Elle est alors disponible à cette fonction.

Les fonctions(9):

Exemple:

```
int main(){
int somme(int , int, int);// déclaration locale
int a,b,c,s;
printf("\n donner la valeur de a ");
scanf("%d",&a);
printf("\n donner la valeur de b ");
scanf("%d",&b);
printf("\n donner la valeur de c ");
scanf("%d",&c);
s=somme(a,b,c);
printf("La somme est %d",s);
    return 0;
}
int somme(int a,int b,int c){
    int s;
    s=a+b+c;
    return s;}
```



```
 donner la valeur de a 7

 donner la valeur de b 4

 donner la valeur de c 4
La somme est 15
Process returned 0 (0x0)   execution time : 5.518 s
Press any key to continue.
```

Les fonctions(10):

Déclaration d'une fonction:

Déclaration globale:

Une fonction peut être déclarée globalement **au début du programme** (derrière les instructions #**include**). Elle est alors disponible à toutes les fonctions du programme.

Exemple:

```
int somme(int , int, int);// déclaration globale
int main(){
int a,b,c,s;
printf("\n donner la valeur de a ");
scanf("%d",&a);
printf("\n donner la valeur de b ");
scanf("%d",&b);
printf("\n donner la valeur de c ");
scanf("%d",&c);
s=somme(a,b,c);
printf("La somme est %d",s);
    return 0;
}
int somme(int a,int b,int c){
    int s;
    s=a+b+c;
    return s;}
```

Les fonctions(11):

Type d'une fonction:

1) Pas de retour et pas d'arguments

```
Void nom_fonction()
{
Instructions;
}
```

2) Pas de retour et avec arguments

```
Void nom_fonction(type arg1,…)
{
Instructions;
}
```

3) Avec retour et pas d'arguments

```
Type_retour nom_fonction()
{
Instructions;
Return resultat;
}
```

4) Avec retour et avec arguments

```
Type_retour nom_fonction( type arg1,… )
{
Instructions;
Return resultat;
}
```

Les fonctions(12):

Type d'une fonction:

- Si une fonction **F** fournit un résultat du type **T**, on dit que '**la fonction F est du type T**'.
- Si une fonction ne fournit pas de résultat, il faut indiquer **void** (vide) comme **type du résultat**.
- Si une fonction n'a pas de paramètres, on peut déclarer la liste des paramètres comme **(void)** ou simplement comme **()**.
- **Le type par défaut est int**: si le type d'une fonction n'est pas déclaré explicitement, elle est automatiquement du type **int**.
- **L'imbrication de fonctions n'est pas autorisée**: une fonction ne peut pas être déclarée à l'intérieur d'une autre fonction. **Par contre, une fonction peut appeler une autre fonction**. Cette dernière doit être déclarée avant celle qui l'appelle.

Les fonctions(13):

Type d'une fonction:

Exemple: Fonction puissance

Pas de retour et avec arguments:

```
Void puissance(int n){
Int p;
P=pow(n,n);
Printf("la puissance de %d est %d:",n,p);
}
```

Avec de retour et avec arguments:

```
int puissance(int n){
Int p;
P=pow(n,n);
Printf("la puissance de %d est %d:",n,p);
Return P;
}
```

Les fonctions(14):

Appel d'une fonction:

1) Pas de retour et pas d'arguments

```
Int main()
{
nom_fonction();
}
```

2) Pas de retour et avec arguments

```
Int main()
{
nom_fonction(arg1, arg2…);
}
```

3) Avec retour et pas d'arguments

```
Int main()
{
X=nom_fonction();
}
```

4) Avec retour et avec arguments

```
Int main()
{
X=nom_fonction( arg1,arg2… );
}
```

Les fonctions(15):

Fonction Vs procédure:

Les fonctions:

- Une **fonction** dispose de **0 ou plusieurs** arguments qui correspondent à des informations qui lui sont transmises et elle fournit **un unique résultat**.
- **Une fonction** peut apparaitre dans une expression. **(exemple: g=somme(x,y,z)+9);**

Les procédures:

- Le **langage C** ne comporte pas à strictement parler le concept de procédure.
- **La procédure** ne possède pas une valeur de retour, son appel ne peut pas apparaitre dans une expression.
- Une procédure dispose d'arguments. Parmi les arguments, certains sont des informations qui lui sont transmises, d'autres, sont des informations qu'elle produit en retour de son appel.

le programmeur peut réaliser une procédure à l'aide d'une fonction qui ne rendra aucune valeur.
=> **En langage C, une fonction désigne à la fois une procédure et une fonction.**

Les fonctions(16):

Règles de visibilité des variables:

- **Variables locales, variables globales**
  - **Variables locales:**

Une variable peut être **déclarée localement** dans la fonction qui l'appelle. Elle est alors **disponible à l'intérieur de cette fonction**.

Exemple:

```
main() {
int A;
}

void fonction()
{
int A;
 }
```

=> Elle est donc visibles, valides et utilisables uniquement pour ces blocs ou fonctions.

Les fonctions(17):

Règles de visibilité des variables:

- **Variables locales, variables globales**
  - **Variables globales:**

Une variable peut être **déclarée globalement** au début du programme (derrière les instructions #include). Elle est alors **disponibles à toutes les fonctions du programme**.

Exemple:

```
# include <stdio.h>
# include <stdlib.h>
int A;
main() {
……
}
```

Les fonctions(18):

Le passage de paramètre:

- Les paramètres d'une fonction sont simplement des **variables locales** qui sont initialisées par les **valeurs** obtenues lors de l'appel.

❖ Conversion automatique:

- Lors d'un appel, le nombre et l'ordre des paramètres doivent nécessairement correspondre aux indications de la déclaration de la fonction. Les paramètres sont automatiquement convertis dans les types de la déclaration avant d'être passés à la fonction.

❖ Exemple:

- Le prototype de la fonction **pow** (bibliothèque <**math**>) est déclaré comme suit:

double pow (double, double);

Au cours des instructions,

```
int A, B;
...
A = pow (B, 2);
```

Les fonctions(19):

Le passage de paramètre:

❖ Exemple:

=> Nous assistons à trois conversions automatiques:

- ✓ Avant d'être transmis à la fonction, la valeur de **B** est convertie en **double**.
- ✓ La valeur **2** est convertie en **2.0** .
- ✓ Comme **pow** est du type **double**, le résultat de la fonction doit être converti en **int** avant d'être affecté à **A**.

Passage des paramètres par valeur:

- En C, le passage des paramètres se fait toujours par la valeur, c.-à-d. les fonctions n'obtiennent que les valeurs de leurs paramètres et n'ont pas d'accès aux variables elles-mêmes.
- Les paramètres d'une fonction sont à considérer comme des variables locales qui sont initialisées automatiquement par les valeurs indiquées lors d'un appel.
- A l'intérieur de la fonction, nous pouvons donc changer les valeurs des paramètres sans influencer les valeurs originales dans les fonctions appelantes.

Les fonctions(20):

Le passage de paramètre:

Passage des paramètres par valeur:

❖ Exemple:

- La fonction ETOILES dessine une ligne de **N étoiles**. Le paramètre **N est modifié à l'intérieur de la fonction:**

```
void ETOILES(int N)
 {
while (N>0) {
 printf("*");
N--;
}
printf("\n");
}
```

En utilisant N comme compteur.

Les fonctions(21):

Le passage de paramètre:

Passage des paramètres par valeur:

- **Exemple:**

La fonction **TRIANGLE**, appelle la fonction **ETOILES** en utilisant la variable **L** comme paramètre:

```
void TRIANGLE(void)
{
int L;
for (L=1; L<10; L++)
ETOILES(L);
}
```

Au moment de l'appel, la **valeur** de **L est copiée dans N**. La variable N peut donc être **décrémentée à l'intérieur de ETOILES, sans influencer la valeur originale de L**.

Les fonctions(22):

Le passage de paramètre:

Passage des paramètres par valeur:

- ❖ Exemple:

Triangle L		Etoilles N
1	appel →	1
1	← retour	0
2	→	2
		1
2	←	0
3	→	3
		2
		1
3	←	0

```
void
TRIANGLE(void)
{
int L;
for (L=1; L<10; L++)
ETOILES(L);
}
```

```
void
ETOILES(int
N)
 {
while (N>0) {
 printf("*");
N--;
}
printf("\n");
}
```

Les fonctions(23):

Le passage de paramètre:

Passage des paramètres par adresse:

- Comme nous l'avons constaté ci-dessus, une fonction n'obtient que les valeurs de ses paramètres.
- **Pour changer la valeur d'une variable de la fonction appelante**, nous allons procéder comme suit:
  - la fonction appelante doit **fournir l'adresse de la variable** et
  - la fonction appelée doit **déclarer le paramètre comme pointeur**.

On peut alors atteindre la variable à l'aide du pointeur.

Exemple:

Nous voulons écrire une fonction **PERMUTER** qui échange le contenu de deux variables du type **int**. En première approche, nous écrivons la fonction suivante:

```
void PERMUTER (int A, int B)
{ int aide;
aide= A;
A = B;
B = aide; }
```

Les fonctions(24):

Le passage de paramètre:

Passage des paramètres par adresse:

Exemple:

Nous appelons la fonction pour deux variables X et Y par:

PERMUTER(X, Y);

Résultat: X et Y restent inchangés !

Explication: Lors de l'appel, les **valeurs** de X et de Y sont copiées dans les paramètres A et B. PERMUTER échange bien contenu des variables **locales** A et B, mais les valeurs de X et Y restent les mêmes.

Appel	
X	Y
3	4
3	4

Appel →

← Retour

Permuter		
A	B	aide
3	4	
4	3	3

Les fonctions(25):

Le passage de paramètre:

Passage des paramètres par adresse:

Exemple:

Pour pouvoir modifier le contenu de X et de Y, la fonction **PERMUTER** a besoin des adresses de X et Y. En utilisant des pointeurs, nous écrivons une deuxième fonction:

```
void PERMUTER (int *A, int *B)
{ int AIDE;
AIDE = *A;
*A = *B;
*B = AIDE; }
```

Nous appelons la fonction par:

```
PERMUTER(&X, &Y);
```

Résultat: Le contenu des variables X et Y est échangé !

Explication: Lors de l'appel, les **adresses** de X et de Y sont copiées dans les **pointeurs** A et B. **PERMUTER** échange ensuite le contenu des adresses indiquées par les pointeurs A et B.

Les fonctions(26):

Le passage de paramètre:

Passage des paramètres par adresse:

Exemple:

Appel	
X	Y
3	4
4	3

Appel →

← Retour

Permuter		
*A	*B	aide
3	4	
4	3	3

Les pointeurs(1):

Introduction:

- La plupart des langages de programmation offrent la possibilité d'accéder aux données dans la mémoire de l'ordinateur à l'aide de **pointeurs**, c.-à-d. à l'aide de variables auxquelles on peut attribuer les **adresses d'autres variables**.
- les pointeurs nous permettent d'écrire des programmes plus compacts et plus efficients et fournissent souvent la seule solution raisonnable à un problème. Ainsi, la majorité des applications écrites en C profitent extensivement des pointeurs.

Définition:

- Un **pointeur** est une variable spéciale qui peut contenir l'**adresse** d'une autre variable.
- En C, chaque pointeur est limité à un type de données. Il peut contenir l'adresse d'une variable simple de ce type ou l'adresse d'une composante d'un tableau de ce type.

➔ Si un pointeur P contient l'adresse d'une variable A, on dit que **'P pointe sur A'**.

Les pointeurs(2):

Remarque:

Les pointeurs et les noms de variables ont le même rôle: Ils **donnent accès à un emplacement dans la mémoire interne** de l'ordinateur. Il faut quand même bien faire la différence:

* Un **pointeur** est une variable qui peut '**pointer**' sur différentes adresses.
* Le **nom d'une variable** reste toujours lié à la même adresse.

Les opérateurs de base:

Lors du travail avec les pointeurs, nous avons besoin:

✓ &: pour obtenir l'adresse d'une variable.
✓ * : pour accéder au contenu d'une adresse.

Les pointeurs(3):

Exercice:

Instruction	A	B	C	P1	P2
Initialisation	1	2	3	-	-
P1=&A;					
P2=&C;					
*P1=(*P2)++;					
P1=P2;					
P2=&B;					
*P1-=*P2;					
++*P2;					
P1=*P2;					
A=++*P2* *P1;					
P1=&A;					
*P2=*P1/=*P2;					

Les pointeurs(4):

Pointeurs et tableaux:

- En C, il existe une relation très étroite entre tableaux et pointeurs.
- Comme nous l'avons déjà constaté au chapitre 3, le nom d'un tableau représente l'adresse de son premier élément. En d'autre termes: **&tableau[0] et tableau** sont une seule et même adresse.
- En simplifiant, nous pouvons retenir que **le nom d'un tableau** est un **pointeur constant** sur le premier **élément du tableau**.

Exemple:

En déclarant un tableau A de type **int** et un pointeur P sur **int**,

int A[10];

int *P;

l'instruction: **P = A;** est équivalente à **P = &A[0];**



Les pointeurs(5):

Pointeurs et tableaux:

Si P pointe sur une composante quelconque d'un tableau, alors **P+1** pointe sur la composante suivante.

Plus généralement:

- **P+i:** pointe sur la i-ième composante derrière P.
- **P-i:** pointe sur la i-ième composante devant P.

➔ chaque pointeur est limité à un seul type de données.

Résumons: Soit un tableau A d'un type quelconque et i un indice pour les composantes de A, alors

A: désigne l'adresse de **A[0]**

A+i désigne l'adresse de **A[i]**

***(A+i)** désigne le contenu de **A[i]**

Si **P = A**, alors

P pointe sur l'élément **A[0]**

P+i pointe sur l'élément **A[i]**

***(P+i)** désigne le contenu de **A[i]**

Les pointeurs(6):

Arithmétique des pointeurs:

- Affectation par un pointeur sur le même type

- Soient P1 et P2 deux pointeurs sur le même type de données, alors l'instruction

  P1 = P2; ➔ fait pointer P1 sur le même objet que P2

- Addition et soustraction d'un nombre entier

- Si P pointe sur l'élément A[i] d'un tableau, alors

  P+n; pointe sur A[i+n]

  P-=n; P pointe sur A[i-n]

- Incrémentation et décrémentation d'un pointeur

- Si P pointe sur l'élément A[i] d'un tableau, alors après l'instruction

  P++; P pointe sur A[i+1]

  P--; P pointe sur A[i-1]

Les pointeurs(7):

- Soustraction de deux pointeurs

- Soient P1 et P2 deux pointeurs qui pointent **dans le même tableau**.
- **P1-P2** fournit le nombre de composantes comprises entre P1 et P2.
- Le résultat de la soustraction **P1-P2** est
  - **négatif**, si P1 précède P2
  - **zéro**, si P1 = P2
  - **positif**, si P2 précède P1
  - **indéfini**, si P1 et P2 ne pointent pas dans le même tableau

⇒ Plus généralement, la soustraction de deux pointeurs qui pointent dans le même tableau est équivalente à **la soustraction des indices** correspondants.

Remarque:

L'addition, la soustraction, l'incrémentation et la décrémentation sur les pointeurs sont seulement définies à l'intérieur d'un tableau. Si l'adresse formée par le pointeur et l'indice sort du domaine du tableau, alors le résultat n'est pas défini.

Les pointeurs(8):

- Comparaison de deux pointeurs

- On peut comparer deux pointeurs par <, >, <=, >=, ==, !=.
- La comparaison de deux pointeurs qui pointent **dans le même tableau** est équivalente à la comparaison des indices correspondants.

Exercice:

Soit **P** un pointeur qui **'pointe'** sur un tableau **A**:

```
int A[] = {12, 23, 34, 45, 56, 67, 78, 89, 90};
 int *P;
P = A;
```

Quelles valeurs ou adresses fournissent ces expressions:

a) *P+2

b) *(P+2)

c) &P+1

d) &A[4]-3

e) A+3

f) &A[7]-P

g) P+(*P-10)

h) *(P+*(P+8)-A[7])

Les pointeurs(8):

-Pointeurs sur char et chaînes de caractères constantes:

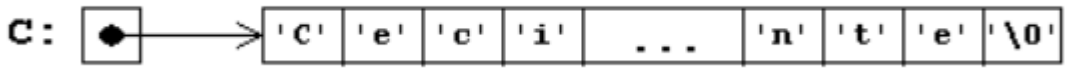
Affectation:

- On peut attribuer **l'adresse d'une chaîne de caractères constante** à un pointeur sur **char**:

Exemple:

char *C;

C = "Ceci est une chaîne de caractères constante";



- Nous pouvons lire cette chaîne constante (**p.ex: pour l'afficher**), mais il n'est pas recommandé de la modifier.

Initialisation:

- Un pointeur sur **char** peut être initialisé lors de la déclaration si on lui affecte l'adresse d'une chaîne de caractères constante:

char *B = "Bonjour !";

Les pointeurs(9):

-Pointeurs sur char et chaînes de caractères constantes:

Initialisation:

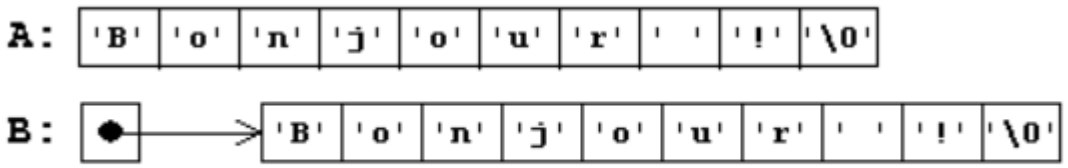
Attention :

Il existe une différence importante entre les deux déclarations:

char A[] = "Bonjour !"; // un tableau

char *B = "Bonjour !"; // un pointeur

- ✓ **A est un tableau** qui a exactement la grandeur pour contenir la chaîne de caractères et la terminaison '\0'. Les caractères de la chaîne peuvent être changés, mais le nom A va toujours pointer sur la même adresse en mémoire.
- ✓ **B est un pointeur** qui est initialisé de façon à ce qu'il pointe sur une chaîne de caractères constante stockée quelque part en mémoire. Le pointeur peut être modifié et pointer sur autre chose. La chaîne constante peut être lue, copiée ou affichée, **mais pas modifiée**.



Les pointeurs(10):

-Pointeurs sur char et chaînes de caractères constantes:

- Si nous affectons une nouvelle valeur à un pointeur sur une chaîne de caractères constante, nous **risquons de perdre la chaîne constante**.
- D'autre part, un pointeur sur **char** a l'avantage de pouvoir pointer sur des chaînes de n'importe quelle longueur.

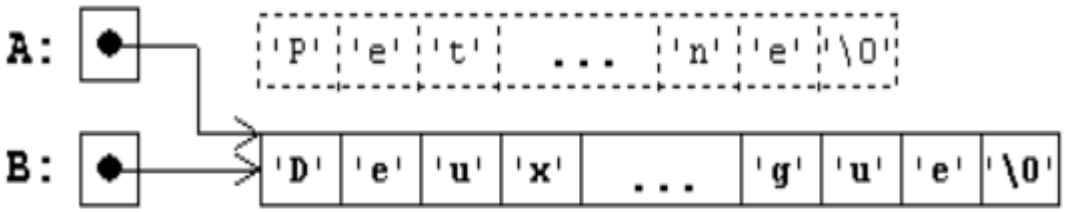
Exemple:

char *A = "Petite chaîne";

char *B = "Deuxième chaîne un peu plus longue";

A = B;

➔Maintenant A et B pointent sur la même chaîne; la "**Petite chaîne**" est perdue.



Les pointeurs(11):

- Pointeurs et tableaux à deux dimensions:

Exemple:

- Le tableau M à deux dimensions est défini comme suit:

int M[4][10] = {{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, {10,11,12,13,14,15,16,17,18,19}, {20,21,22,23,24,25,26,27,28,29}, {30,31,32,33,34,35,36,37,38,39}};

- Le nom du tableau M représente l'adresse du premier élément du tableau et pointe sur le **tableau M[0]** qui a la valeur: **{0,1,2,3,4,5,6,7,8,9}.**

➔L'expression **(M+1)** est l'adresse du deuxième élément du tableau et pointe sur **M[1]** qui a la valeur: **{10,11,12,13,14,15,16,17,18,19}**

- Comme nous avons vu, un tableau à deux dimensions est un tableau unidimensionnel dont chaque composante est un tableau unidimensionnel.

Les pointeurs(12):

Pointeurs et tableaux à deux dimensions:

- Ainsi, le premier élément de la matrice M est le **vecteur {0,1,2,3,4,5,6,7,8,9}**, le deuxième élément est **{10,11,12,13,14,15,16,17,18,19}** et ainsi de suite.

➔ L'arithmétique des pointeurs qui respecte automatiquement les dimensions des éléments conclut logiquement que: **M+I** désigne l'adresse du tableau **M[I]**

Pour accéder à l'aide de pointeurs aux éléments de chaque composante du tableau, c.à-d.: aux éléments **M[0][0], M[0][1], ... , M[3][9].** Il consiste à convertir la valeur de M (qui est un pointeur sur **un tableau du type int**) en un pointeur sur **int**. On pourrait se contenter de procéder ainsi:

Les pointeurs(13):

Pointeurs et tableaux à deux dimensions:

int M[4][10] = {{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, {10,11,12,13,14,15,16,17,18,19},
{20,21,22,23,24,25,26,27,28,29}, {30,31,32,33,34,35,36,37,38,39}};
Int *P;
P=M; // conversion automatique
Ou bien **P=(int *)M // conversion forcée.**

- Cette dernière affectation entraîne une conversion automatique de l'adresse **&M[0]** dans l'adresse **&M[0][0].** (Remarquez bien que l'adresse transmise reste la même, seule la nature du pointeur a changé).

Les pointeurs(14):

Tableaux de pointeurs:

Déclaration d'un tableau de pointeurs:

<Type> *<Nom_Tab>[<n>]

➔déclare un tableau <**Nom_Tab**> de <**n**> pointeurs sur des données du type <**Type**>.

Exemple

double *A[10];

➔déclare un tableau de 10 pointeurs sur des rationnels du type **double** dont les adresses et les valeurs ne sont pas encore définies.

Remarque:

Le plus souvent, les tableaux de pointeurs sont utilisés pour mémoriser de façon économique des **chaînes de caractères de différentes longueurs**. Dans la suite, nous allons surtout considérer les tableaux de pointeurs sur des chaînes de caractères.

Les pointeurs(15):

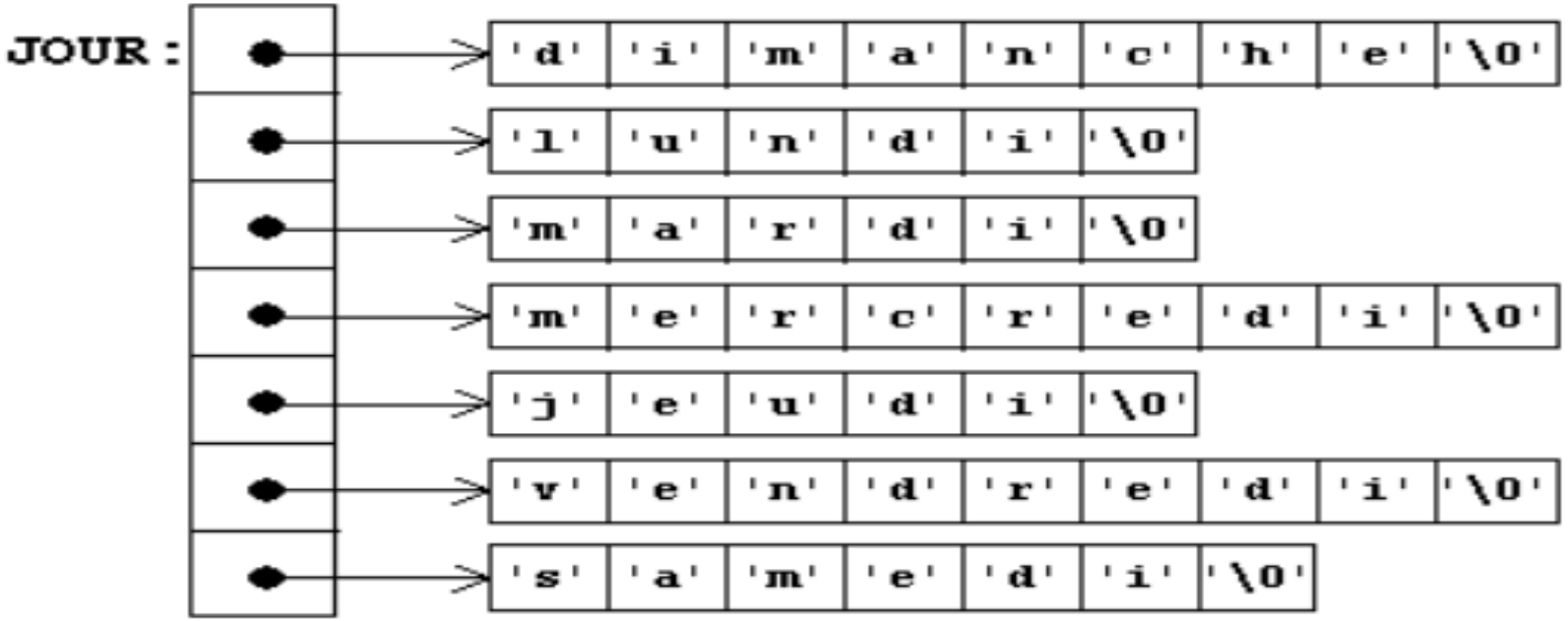
Initialisation:

Nous pouvons initialiser les pointeurs d'un tableau sur **char** par les adresses de chaînes de caractères constantes.

Exemple:

char *jour[] = {"dimanche", "lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi"};

déclare un tableau **jour[]** de 7 pointeurs sur **char**. Chacun des pointeurs est initialisé avec l'adresse de l'une des 7 chaînes de caractères.



Les pointeurs(16):

- **Tableaux de pointeurs:**

✓ On peut afficher les 7 chaînes de caractères en fournissant les adresses contenues dans le tableau **jour** à **printf** (ou **puts**) :

```
int I;
for (I=0; I<7; I++)
printf("%s\n", jour[I]);
```

✓ Comme **jour[I]** est un pointeur sur **char**, on peut afficher les premières lettres des jours de la semaine en utilisant l'opérateur 'contenu de' :

```
int I;
for (I=0; I<7; I++)
printf("%c\n", *jour[I]);
```

✓ L'expression **jour[I]+J** désigne la **J-ième** lettre de la **I-ième** chaîne. On peut afficher la troisième lettre de chaque jour de la semaine par:

```
int I;
for (I=0; i<7; I++)
printf("%c\n",*(jour[I]+2));
```

Les pointeurs(17):

Tableaux de pointeurs:

Résumons: Les tableaux de pointeurs int *D[];
déclare **un tableau de pointeurs** sur des éléments du type **int**
D[i] peut pointer sur des variables simples ou sur les composantes d'un tableau.
D[i] désigne l'adresse contenue dans l'élément i de D (Les adresses dans D[i] sont variables)
***D[i]** désigne le contenu de l'adresse dans D[i]

Les pointeurs(18):

Allocation dynamique de mémoire:

- Nous avons vu que l'utilisation de pointeurs nous permet de mémoriser économiquement des données de différentes grandeurs.
- Si nous générons ces données pendant l'exécution du programme, il nous faut des moyens pour réserver et libérer de la mémoire au fur et à mesure que nous en avons besoin.

Déclaration statique de données:

- Chaque variable dans un programme a besoin d'un certain nombre d'octets en mémoire. Jusqu'ici, la réservation de la mémoire s'est déroulée automatiquement par l'emploi des déclarations des données.

Exemples:

```
float A, B, C; // réservation de 12 octets
short D[10][20]; // réservation de 400 octets
char E[] = {"Bonjour !"}; // réservation de 10 octets
char F[][10] = {"un", "deux", "trois", "quatre"}; // réservation de 40 octets
```

Les pointeurs(19):

Allocation dynamique de mémoire:

Déclaration statique de données:

Exemples:

Pointeurs:

```
double *G; // réservation de p octets
float *I[10]; // réservation de 10*p octets
char *J = "Bonjour !"; // réservation de p+10 octets
```

➔ Souvent, nous devons travailler avec des données dont nous ne pouvons pas prévoir le nombre et la grandeur lors de la programmation.

➔ Ce serait alors un gaspillage de réserver toujours l'espace maximal prévisible. Il nous faut donc un moyen de gérer la mémoire lors de l'exécution du programme.

Les pointeurs(19):

Allocation dynamique de mémoire:

La fonction malloc et l'opérateur sizeof:

- La fonction **malloc** de la bibliothèque <**stdlib**> nous aide à localiser et à réserver de la mémoire au cours d'un programme.

La fonction malloc:

- **malloc(<N>)** fournit l'adresse d'un bloc en mémoire de **<N> octets libres** ou la valeur zéro s'il n'y a pas assez de mémoire.

Exemple:

- Supposons que nous ayons besoin d'un bloc en mémoire pour un **texte de 4000 caractères**. Nous disposons d'un pointeur T sur **char** (**char *T**). Alors l'instruction:

T = malloc(4000);

➔ fournit l'adresse d'un bloc de 4000 octets libres et l'affecte à T. S'il n'y a plus assez de mémoire, T obtient la valeur zéro.

Les pointeurs(20):

Allocation dynamique de mémoire:

La fonction malloc et l'opérateur sizeof:

- Si nous voulons réserver de la mémoire pour des données d'un type dont la grandeur varie d'une machine à l'autre, nous avons besoin de la grandeur effective d'une donnée de ce type.

  ➔L'opérateur **sizeof** nous aide alors à préserver la portabilité du programme.

L'opérateur unaire sizeof:

- ✓ **sizeof <var>** ➔ fournit la grandeur de la variable **<var>**
- ✓ **sizeof <const>** ➔fournit la grandeur de la constante **<const>**
- ✓ **sizeof (<type>)** ➔fournit la grandeur pour un objet du type **<type>**

Exemple:

Nous voulons réserver de la mémoire pour X valeurs du type **int**; la valeur de X est lue au clavier:

```
int X;
 int *PNum;
printf("Introduire le nombre de valeurs :");
scanf("%d", &X);
PNum = malloc(X*sizeof(int));
```

Les pointeurs(20):

Allocation dynamique de mémoire:

La fonction free:

✓ Si nous n'avons plus besoin d'un bloc de mémoire que nous avons réservé à l'aide de **malloc**, alors nous pouvons le libérer à l'aide de la fonction **free** de la bibliothèque <**stdlib**>.

free(<Pointeur>) ➔ libère le bloc de mémoire désigné par le **<Pointeur>;** n'a pas d'effet si le pointeur a la valeur zéro.

Attention !

* La fonction **free** peut aboutir à un désastre si on essaie de libérer de la mémoire qui n'a pas été allouée par **malloc**.

* Si nous ne libérons pas explicitement la mémoire à l'aide **free**, alors elle est libérée automatiquement à la fin du programme.

La récursivité(1):

La fonction récursive en C:

- Une fonction est dite **récursive** si elle est **appelée en elle-même**.
- Voici deux conditions essentielles pour implémenter la récursivité en C:
  - ✓ **Une condition de sortie:** cette condition permet à la fonction d'identifier quand quitter cette fonction. Dans le cas où nous ne spécifions pas la condition de sortie, le code entrera dans une boucle infinie.
  - ✓ **Changer le compteur:** Changer le compteur à chaque appel à cette fonction.

➔ Ces fonctions sont utiles **pour résoudre des problèmes mathématiques** qui nécessitent un processus similaire appelé plusieurs fois.

La récursivité(2):

La fonction récursive en C:

Exemple: la fonction factorielle

- **Solution itérative**

```
int fact(int n){
int f;
if(n==0||n==1)
f=1;
else {
f=1;
for(int i=2; i<=n; i++)
f*=i;
}
return f;
}
```

f=5

Itérations:

F=2*1

F=3*2

F=4*6

F=5*24



Affichage: factorielle de 5= 120

La récursivité(2):

La fonction récursive en C:

solution récursive:

F(1)=1
F(2)=2*1➔ f(2)=2*f(1)
F(3)=3*2*1➔ f(3)=3*f(2)
F(4)=4*3*2*1➔ f(4)=4*f(3)
F(5)=5*4*3*2*1➔ f(5)=5*f(4)



F(n)=n*f(n-1)

```
int factoriel(int n){
if(n==0|| n==1)
retern 1;
else
return n*factoriel(n-1);
}
```

Factoriel(n)= { 1 si n=0 ou n=1 ; N*factoriel(n-1) si n>1

La récursivité(2):

La fonction récursive en C:

solution récursive:

Factoriel(1) Factoriel(2) Factoriel(3) Factoriel(4) Factoriel(5)	1 2*Factoriel(1)=2 3*Factoriel(2)=6 4*Factoriel(3)=24 5*Factoriel(4)=120	➔ Affichage : factoriel (5)=120

- RAM